

Ayudantía 1

SISTEMAS DISTRIBUIDOS

PROFESOR: NICOLAS HIDALGO

AYUDANTE CÁTEDRA: BENJAMIN ACEITUNO, VICENTE DÍAZ

AYUDANTE CORRECTOR: ISIDORA GONZALEZ, NATALIA ORTEGA



Contacto

Benjamín Aceituno

benjamin.aceituno@mail.udp.cl

Vicente Díaz

Vicente.diaz5@mail.udp.cl

Isidora González

isidora.gonzalez4@mail.udp.cl

Natalia Ortega

natalia.ortega1@mail.udp.cl



Si estas en windows necesario instalar WSL(compilador de linux):

1) Buscar Windows PowerShell y ejecutar como administrador

2) Ejecutar el comando: `wsl --install`

(si hay un error se debe activar la virtualizacion desde la BIOS)

<https://support.microsoft.com/es-es/windows/habilitar-la-virtualización-en-windows-c5578302-6e43-4b4b-a449-8ced115f58e1>

3) Reiniciar PC.

4) Una vez reiniciado se abrirá una consola de ubuntu donde tendran que crear un usuario y contraseña.

5) Entrar a Panel de control < Desinstalar un programa < Activar o desactivar características de Windows

Una vez en este lugar buscar "Subsistema de Linux para Windows" y activarlo.

6) Descargar Docker (Si es windows, opcion AMD64)(Si es linux empezar tutorial desde aquí)

<https://www.docker.com>

7) Reiniciar.

8) Una vez instalado todo comprobar que todo este instalado correctamente.

Abrir WindowsPowerShell y ejecutar: `wsl --status`

Si les aparece un mensaje asi es correcto ya que docker ocupa la version 2:

Default Distribution: Ubuntu

Default Version: 2

9) Ahora para comprobar el funcionamiento de Docker deben entrar a WSL:

`docker --version`

Si les aparece un mensaje asi es correcto:

Docker version 25.0.2, build ...

Si les aparece un mensaje asi:

The command 'docker' could not be found in this WSL 2 distro.

Deben ir a Docker Desktop < Settings < Resources < WSL integration < Enable integration with additional distros < Ubuntu: ON

Por último ejecutar: `docker run hello-world`

Si ven el mensaje: Hello from Docker!

Está instalado correctamente.

Tutorial instalación Docker (Windows)

```

1  version: '3.8'
2
3  services:
4    postgres:
5      image: postgres:15
6      restart: always
7      ports:
8        - "5433:5432"
9      environment:
10       POSTGRES_USER: postgres
11       POSTGRES_PASSWORD: postgres
12       POSTGRES_DB: yahoo_dataset
13      volumes:
14        - postgres_data:/var/lib/postgresql/data
15        - ./init_db.sql:/docker-entrypoint-initdb.d/init_db.sql
16      healthcheck:
17        test: ["CMD-SHELL", "pg_isready -U postgres -d yahoo_dataset"]
18        interval: 5s
19        timeout: 5s
20        retries: 5
21      networks:
22        - yahoo_network
23
24    pgadmin:
25      image: dpage/pgadmin4
26      restart: always
27      ports:
28        - "5050:80"
29      environment:
30       PGADMIN_DEFAULT_EMAIL: admin@admin.com
31       PGADMIN_DEFAULT_PASSWORD: admin
32      volumes:
33        - pgadmin_data:/var/lib/pgadmin
34      depends_on:
35        - postgres
36      networks:
37        - yahoo_network

```

Para levantar todos los dockers en la carpeta raíz se debe ejecutar:

(Levantamiento lento)

En caso de que sea necesario reconstruir las imagenes de cada servicio ya que hubo un cambio en el Dockerfile o cambiaste las dependencias o quieres asegurarte de que la imagen este actualizada, utilizar:

```
docker-compose up --build
```

(Levantamiento rápido)

En caso de que modifiques el código fuente y estas desarrollando, lo mejor es levantar contenedores ya creadas en tu máquina, no recompila gracias a los volúmenes, utilizar:

```
docker-compose up
```

Para visualizar si los dockers están funcionando comprobarlo con:

```
docker ps
```

```
o
```

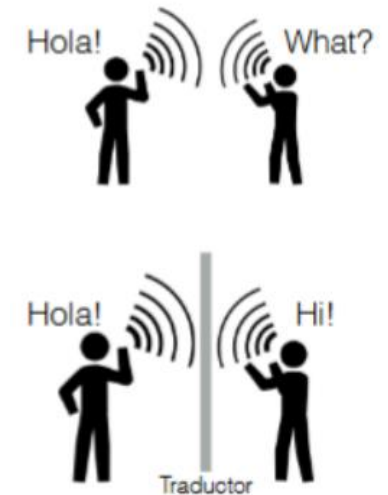
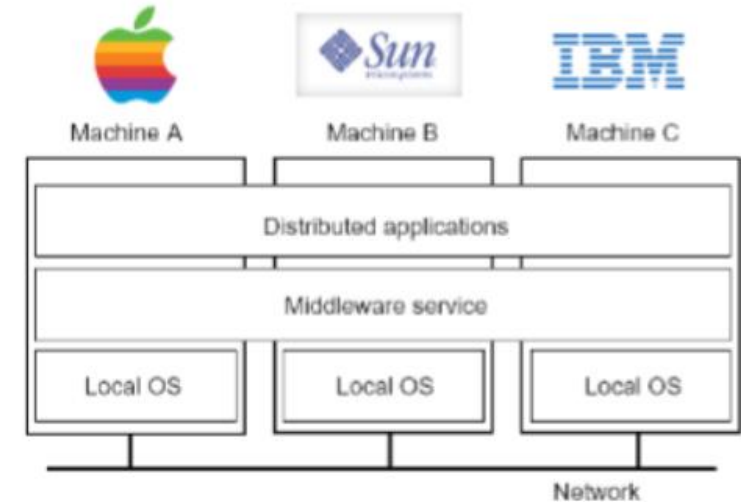
```
docker ps -a
```

Utilización de Docker

← docker-compose.yml

¿Qué son los sistemas distribuidos?

- Conjunto de máquinas que coordinan sus acciones y que aparecen al usuario como sistema único y coherente.
- Middleware: SD emplea software sobre la red para lograr cometido.
- Red de computadores: usuarios interactúan directamente con las máquinas.



¿Porqué y cuándo distribuir?

- Lograr escalabilidad, disponibilidad, eficiencia y cercanía al usuario.
- Supera limitaciones de un sistema centralizado.
- Dependencia al tamaño e infraestructura.



Ventajas:

- Compartir recursos.
- Mejora de desempeño.
- Mayor adaptabilidad.
- Mayor disponibilidad.
- Escalabilidad.

Desafíos:

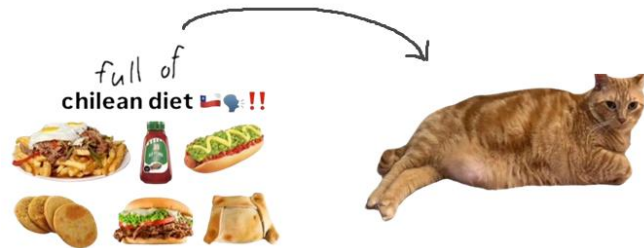
- Heterogeneidad del hardware.
- Consistencia de la interacción.
- Siempre disponible.
- Usuarios y apps no deben notar caídas.
- Seguridad



Objetivos de diseño

Existen 4 objetivos al diseñar un sistema distribuido:

- 1) Poner recursos a disposición.
- 2) Hacer transparente la distribución.
- 3) Ser abierto.
- 4) Escalabilidad.



1) Poner recursos a disposición.

- Facilitar a los usuarios y aplicaciones acceder recursos remotos de manera eficiente.
- Razones para compartir:
 - Económicas.
 - Colaboración e intercambio de información.
- Si se incrementa la conectividad y la compartición de recursos, la seguridad se vuelve relevante.
 - Seguridad de la comunicación.
 - Seguridad de acceso a los datos.



2) Hacer transparente la distribución.

- Esconder el hecho que procesos y recursos están físicamente distribuidos en múltiples computadores.
- Usuarios y aplicaciones ven un sistema único.
- Sistema coherente.



- Transparencia respecto a diferentes ámbitos:
 - Acceso.
 - Ubicación.
 - Migración.
 - Re-localización.
 - Replicación.
 - Concurrencia.
 - Falla.

3) Ser abierto

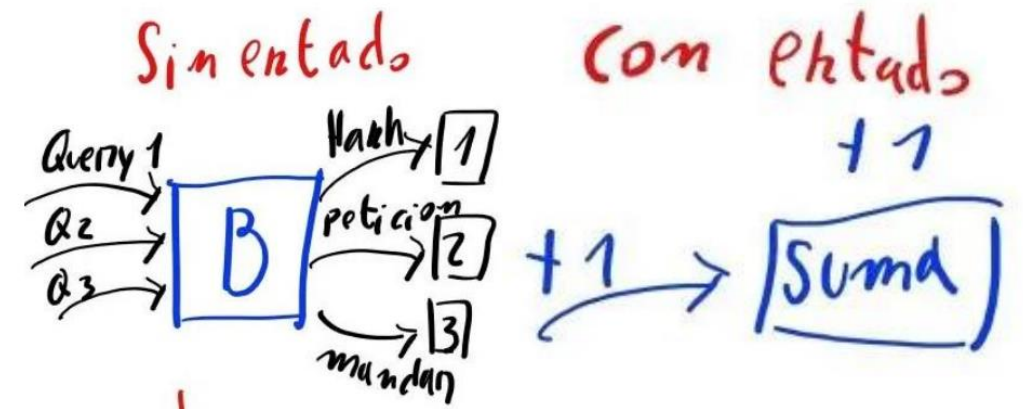
- Sistema distribuido abierto es aquel que ofrece servicios bajo reglas estándar (sintaxis, semántica).
- Interface Definition Language (IDL), lenguaje de especificación de componentes del sistema, como, por ejemplo:
 - Advanced Programming Interface (API).
 - Llamadas a procedimientos remotos (RPC).
- Interoperabilidad: permite que procesos se comuniquen y generen implementaciones diferentes.
- Portabilidad: extensiones pueden ser usadas en otros sistemas
- Fácil extensión: agregar más componentes
 - En la práctica, muchos sistemas son monolíticos (cerrados).



4) Escalabilidad

Existen 2 tipos:

- Escalabilidad Horizontal (scaling out):
Aumentar la cantidad de los recursos.
 - Agregar más nodos de trabajo.
 - Front-End sin estado -> lo que haya pasado antes no influye en las próximas decisiones.
 - Backend con particionamiento, modelo master/slave.



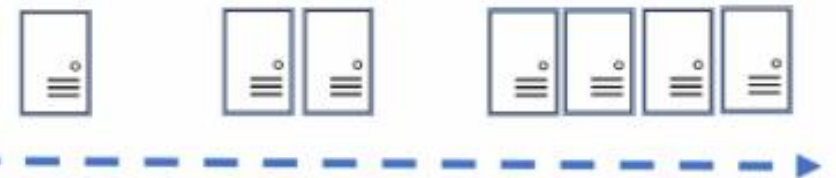
4) Escalabilidad

- Escalabilidad Vertical (scaling up): Aumentar las capacidades de mi recurso
 - Agregar CPU, Memoria, disco.
 - Manejar simultáneo de más conexiones, operaciones, usuarios.
 - Buen I/O y modelos de concurrencia (hebras, asincronismo, etc...).

ESCALAMIENTO VERTICAL
(incrementas tamaño de la instancia CPU, Memoria, etc.)

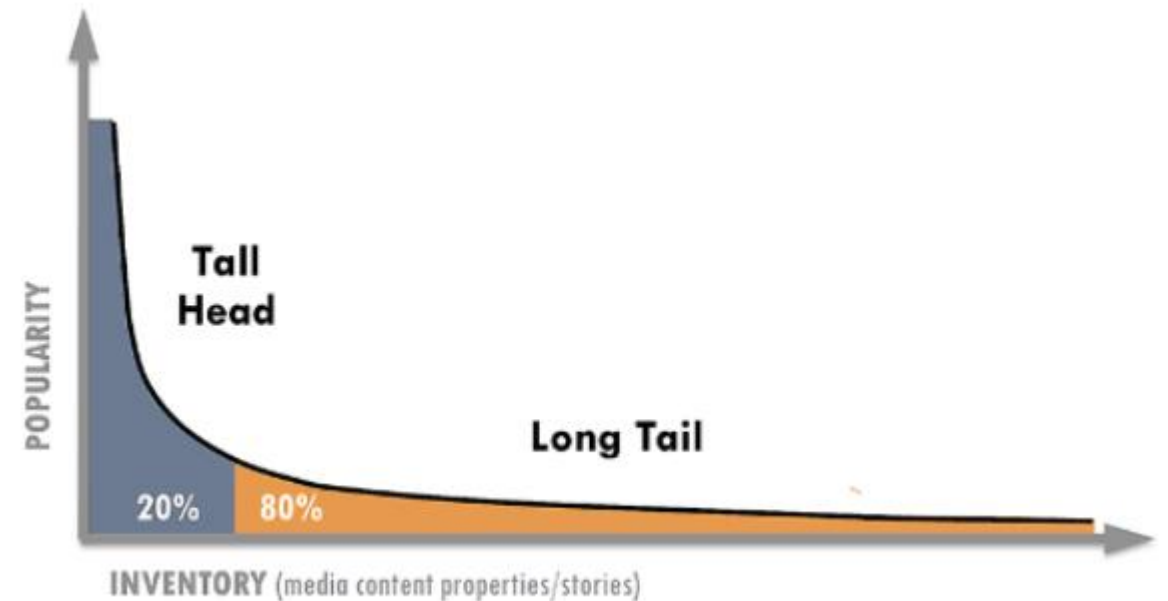


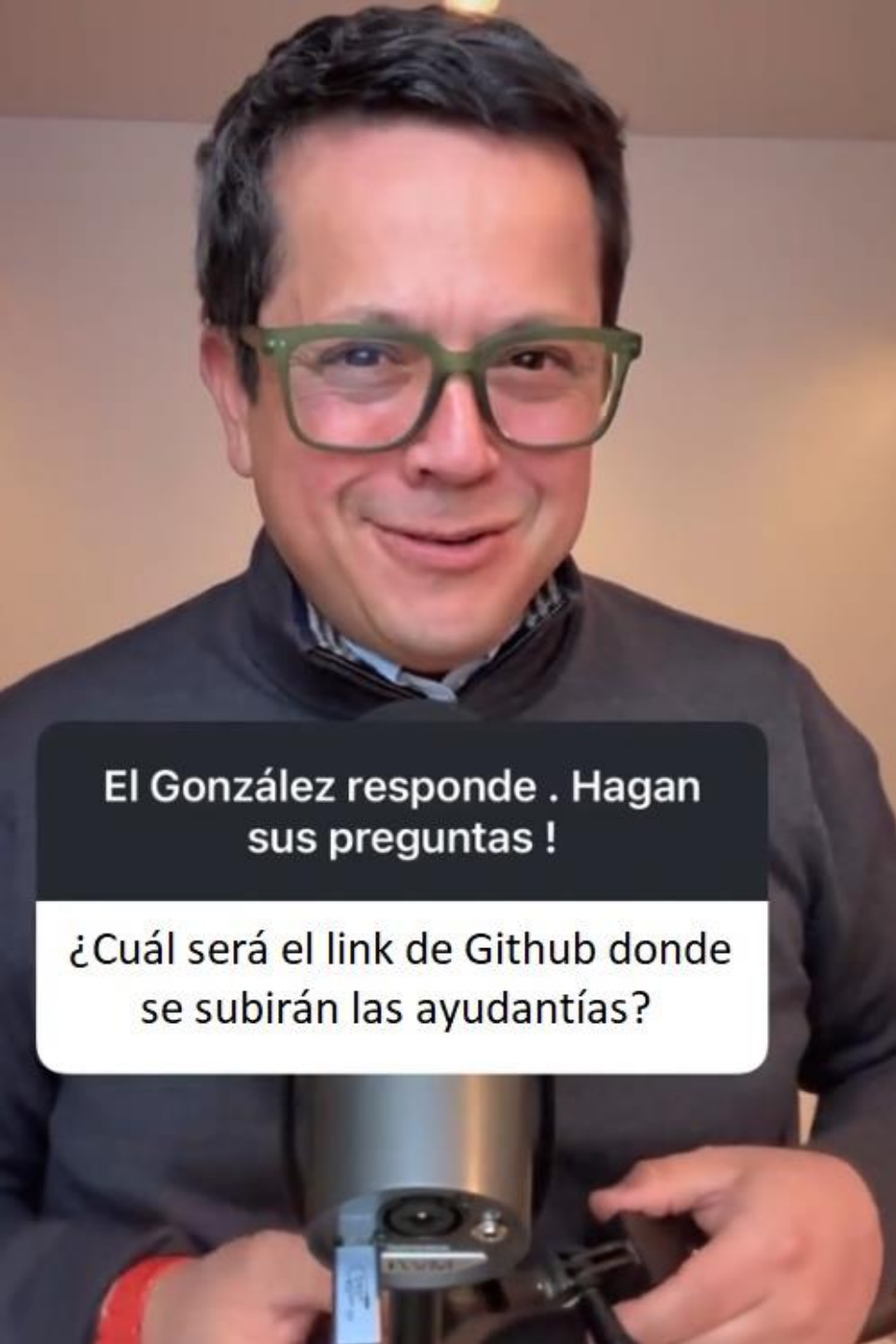
ESCALAMIENTO HORIZONTAL
(Agregas más instancias)



4) Escalabilidad

- Existen 3 técnicas básicas para escalar:
 - I. **Particionar y distribuir:** dividir un componente y distribuirlo, el cuál puede tener 2 variaciones:
 - a. Particionamiento Vertical
 - b. Particionamiento Horizontal (Sharding)
 - II. **Replicar:** hacer copiar de los recursos en diferentes lugares.
 - III. **Cache:** subconjunto de recursos de acceso fácil y rápido.





El González responde . Hagan
sus preguntas !

¿Cuál será el link de Github donde
se subirán las ayudantías?

Link Github

Repositorio:

<https://github.com/benjamp4/Ayudantias-SSDD-2026-2>